

Quick Reference: What to Do Now

Start Here (This Week)

For the CTO / Tech Lead

Today (30 minutes)

- ☐ Schedule Phase 0 kick-off meeting (team + risk lead)
- ☐ Create GitHub repo with this structure:

Issues:

- [Phase 0] Task 0.1: State Machine Document
- [Phase 0] Task 0.2: Architecture Current vs Target
- [Phase 0] Task 0.3: Capital Safety Requirements

- ☐ Assign owners (at least 1 per task)
- ☐ Create shared documentation folder (Confluence/Google Docs)

Monday morning (team meeting)

- ☐ Present 10 inconsistencies overview (use the visualization)
- ☐ Walk through Phase 0 plan week-by-week
- ☐ Set deadline: Friday end-of-day for all Phase 0 docs
- ☐ Clarify: what's current vs. target state?
- ☐ Assign task owners

Monday-Friday

- ☐ Daily standup 10:00 AM (even if brief)
- ☐ Thursday: Risk/CTO review of Phase 0 docs
- ☐ Friday: Sign-off meeting + Phase 1 planning

For the Execution Engineer (owns Task 1.1 + 1.2)

Task 1.1: Hard Kill Switch

Create file: `trading_system/risk/kill_switch.py`

```
python
```

```

from datetime import datetime
from typing import List
from dataclasses import dataclass
import json

@dataclass
class KillEvent:
    timestamp: str
    reason: str
    open_orders: list
    actor: str = "SYSTEM"

class KillSwitch:
    """Hard stop mechanism. Cannot be disabled or re-armed via code."""

    def __init__(self, log_file: str = "/var/log/trading/kill_switch.log"):
        self.killed = False
        self.log_file = log_file
        self.kill_log: List[KillEvent] = []

    def trigger(self, reason: str, open_orders: list = None):
        """Idempotent hard stop."""
        if open_orders is None:
            open_orders = []

        event = KillEvent(
            timestamp=datetime.utcnow().isoformat(),
            reason=reason,
            open_orders=open_orders
        )

        self.killed = True
        self.kill_log.append(event)

        # Write to immutable log
        with open(self.log_file, 'a') as f:
            f.write(json.dumps(event.__dict__) + '\n')

        # Alert
        self._send_pagerduty_alert(reason)
        self._cancel_all_orders()

    def is_live(self) -> bool:
        """Check before any trading operation."""
        if self.killed:
            raise Exception("TRADING_HALTED: Kill switch triggered")

```

```

        return True

    def _send_pagerduty_alert(self, reason: str):
        # TODO: Implement
        pass

    def _cancel_all_orders(self):
        # TODO: Call broker API to cancel all orders
        pass

# Usage:
# if not kill_switch.is_live():
#     return # Don't execute

```

Test file: `tests/test_kill_switch.py`

```

python

import pytest
from trading_system.risk.kill_switch import KillSwitch

def test_kill_switch_triggers():
    ks = KillSwitch()
    ks.trigger(reason="Test trigger", open_orders=[])
    assert ks.killed is True

def test_kill_switch_is_live_throws():
    ks = KillSwitch()
    ks.trigger(reason="Test")
    with pytest.raises(Exception, match="TRADING_HALTED"):
        ks.is_live()

def test_kill_switch_idempotent():
    ks = KillSwitch()
    ks.trigger(reason="First")
    ks.trigger(reason="Second") # Should not error
    assert len(ks.kill_log) == 2

```

Checklist:

- ☐ Code written
 - ☐ Unit tests pass (test idempotency, test exception on `is_live()`)
 - ☐ Log file is actually written to disk
 - ☐ Cannot be re-armed via code (manual restart only)
 - ☐ PagerDuty alert integration (or print for now)
-

Task 1.2: Database Schema

Create file: `migrations/001_initial_schema.sql`

sql

```
-- Core tables for trading system
```

```
-- Orders: current state
```

```
CREATE TABLE orders (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  strategy_id VARCHAR(50) NOT NULL,  
  symbol VARCHAR(10) NOT NULL,  
  side VARCHAR(10) NOT NULL CHECK (side IN ('BUY', 'SELL')),  
  quantity DECIMAL(15,8) NOT NULL,  
  entry_price DECIMAL(15,8) NOT NULL,  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  
  -- Current state  
  state VARCHAR(20) NOT NULL DEFAULT 'CREATE' CHECK (state IN (  
    'CREATE', 'QUEUED', 'SENT', 'ACK', 'PARTIAL_FILL',  
    'FILLED', 'CANCELLED', 'FAILED', 'DEAD_LETTER'  
  )),
```

```
-- Fill tracking
```

```
filled_qty DECIMAL(15,8) DEFAULT 0,  
filled_price DECIMAL(15,8),  
filled_at TIMESTAMPTZ,
```

```
-- Broker linking
```

```
broker_order_id VARCHAR(50),
```

```
-- Error tracking
```

```
error_msg TEXT,
```

```
-- Audit
```

```
updated_at TIMESTAMPTZ DEFAULT NOW(),  
updated_by VARCHAR(50) DEFAULT 'SYSTEM'
```

```
);
```

```
CREATE INDEX idx_orders_symbol ON orders(symbol);
```

```
CREATE INDEX idx_orders_state ON orders(state);
```

```
CREATE INDEX idx_orders_created_at ON orders(created_at);
```

```
-- Immutable event log: every state transition
```

```
CREATE TABLE order_events (  
  id SERIAL PRIMARY KEY,  
  order_id UUID NOT NULL REFERENCES orders(id) ON DELETE CASCADE,  
  event_type VARCHAR(50) NOT NULL, -- CREATED, QUEUED, SENT, ACK, FILLED, FAILED, etc  
  old_state VARCHAR(20),  
  new_state VARCHAR(20),
```

```

-- Event details (flexible)
details JSONB, -- {broker_response, error, retry_attempt, ...}

-- Metadata
created_at TIMESTAMPTZ DEFAULT NOW(),
actor VARCHAR(50) DEFAULT 'SYSTEM',

-- Immutable: no updates
UNIQUE(order_id, created_at)
);

CREATE INDEX idx_order_events_order_id ON order_events(order_id);
CREATE INDEX idx_order_events_created_at ON order_events(created_at);

-- Reconciliation audit
CREATE TABLE reconciliation_runs (
    id SERIAL PRIMARY KEY,
    run_date DATE NOT NULL UNIQUE,

    -- Summary
    system_total_position DECIMAL(15,8),
    broker_total_position DECIMAL(15,8),
    delta DECIMAL(15,8),

    -- Findings
    orphaned_orders JSONB, -- Orders in system but not on broker
    missing_orders JSONB, -- Fills on broker but not in system

    -- Status
    status VARCHAR(20) NOT NULL DEFAULT 'PENDING' CHECK (status IN (
        'PENDING', 'AUTO_RESOLVED', 'MANUAL_REVIEW', 'RESOLVED'
    )),

    -- Notes
    notes TEXT,
    resolved_at TIMESTAMPTZ,
    resolved_by VARCHAR(50),

    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_reconciliation_runs_run_date ON reconciliation_runs(run_date);
CREATE INDEX idx_reconciliation_runs_status ON reconciliation_runs(status);

```

Checklist:

- ☐ Schema file syntax valid (test with `psql -f migrations/001_initial_schema.sql`)
 - ☐ All constraints in place (CHECK, UNIQUE, FK)
 - ☐ Indexes on frequently-queried columns
 - ☐ `order_events` table is write-once (UNIQUE constraint prevents duplicates)
 - ☐ JSONB columns for flexible event details
 - ☐ Audit columns (`created_at`, `updated_by`) on all tables
-

For the Risk Engineer (owns Task 1.3 + oversight)

Task 1.3: Broker Reconciliation Service

Create file: `trading_system/broker/reconciliation.py`

```
python
```

```

from datetime import date
from typing import List, Dict
from dataclasses import dataclass, asdict
import logging

logger = logging.getLogger(__name__)

@dataclass
class ReconciliationResult:
    run_date: date
    system_position: float
    broker_position: float
    delta: float
    orphaned_orders: List[Dict] # In system, not on broker
    missing_fills: List[Dict] # On broker, not in system
    auto_resolved_count: int
    manual_review_count: int
    status: str # OK, AUTO_RESOLVED, MANUAL_REVIEW

class BrokerReconciler:
    def __init__(self, db_connection, broker_api, tolerance: float = 10.0):
        self.db = db_connection
        self.broker = broker_api
        self.tolerance = tolerance # Dollar amount

    def reconcile(self, run_date: date) -> ReconciliationResult:
        """Daily reconciliation."""

        # 1. Get system position
        system_orders = self.db.get_orders_as_of(run_date)
        system_position = self._calculate_position(system_orders)

        # 2. Get broker position
        broker_fills = self.broker.get_fills(run_date)
        broker_position = self._calculate_position(broker_fills)

        delta = abs(system_position - broker_position)

        # 3. Find discrepancies
        orphaned = [] # System has, broker doesn't
        missing = [] # Broker has, system doesn't

        for order in system_orders:
            if not self._find_match(order, broker_fills):
                orphaned.append(order)

```



```

for fill in broker_fills:
    if not self._find_match(fill, system_orders):
        missing.append(fill)

# 4. Categorize
auto_resolved = [o for o in (orphaned + missing) if self._is_minor(o)]
manual_review = [o for o in (orphaned + missing) if not self._is_minor(o)]

# 5. Determine status
if delta < self.tolerance and len(manual_review) == 0:
    status = "OK"
elif len(manual_review) == 0:
    status = "AUTO_RESOLVED"
else:
    status = "MANUAL_REVIEW"

# 6. Log to database
result = ReconciliationResult(
    run_date=run_date,
    system_position=system_position,
    broker_position=broker_position,
    delta=delta,
    orphaned_orders=orphaned,
    missing_fills=missing,
    auto_resolved_count=len(auto_resolved),
    manual_review_count=len(manual_review),
    status=status
)

self.db.save_reconciliation(asdict(result))

# 7. Alert if needed
if status == "MANUAL_REVIEW":
    self._send_alert(f"Reconciliation {run_date}: {len(manual_review)} items need manual review")

logger.info(f"Reconciliation {run_date}: {status} (delta: ${delta:.2f})")
return result

def _calculate_position(self, orders: List) -> float:
    """Sum of all filled quantities * prices."""
    return sum(o['filled_qty'] * o['filled_price'] for o in orders)

def _find_match(self, order: Dict, candidates: List[Dict]) -> Dict:
    """Match on order_id or (symbol, quantity, price)."""
    for candidate in candidates:
        if candidate.get('order_id') == order.get('order_id'):
            return candidate

```

```

        if (candidate.get('symbol') == order.get('symbol') and
            candidate.get('filled_qty') == order.get('filled_qty') and
            abs(candidate.get('filled_price', 0) - order.get('filled_price', 0)) < 0.0001):
            return candidate
    return None

def _is_minor(self, order: Dict) -> bool:
    """Is the delta small enough to auto-resolve?"""
    delta = abs(order.get('delta_amount', 0))
    qty_delta = abs(order.get('delta_qty', 0))
    return delta < self.tolerance and qty_delta < 1

def _send_alert(self, message: str):
    logger.error(message)
    # TODO: Send PagerDuty, Slack, etc.

# Usage:
# reconciler = BrokerReconciler(db, broker_api)
# result = reconciler.reconcile(date.today())
# if result.status == "MANUAL_REVIEW":
#     print(f"Alert: {result.manual_review_count} items need review")

```

Test file: `tests/test_reconciliation.py`

python

```

import pytest
from datetime import date
from trading_system.broker.reconciliation import BrokerReconciler

def test_reconciliation_orphaned_order():
    """Detect order in system but not on broker."""
    reconciler = BrokerReconciler(mock_db, mock_broker)

    mock_db.get_orders_as_of.return_value = [
        {'order_id': '1', 'symbol': 'AAPL', 'filled_qty': 10, 'filled_price': 150}
    ]
    mock_broker.get_fills.return_value = []

    result = reconciler.reconcile(date.today())
    assert len(result.orphaned_orders) == 1
    assert result.status == "MANUAL_REVIEW"

def test_reconciliation_auto_resolve_minor():
    """Auto-resolve small discrepancies."""
    reconciler = BrokerReconciler(mock_db, mock_broker, tolerance=10)

    mock_db.get_orders_as_of.return_value = [
        {'order_id': '1', 'symbol': 'AAPL', 'filled_qty': 10, 'delta_amount': 5}
    ]
    mock_broker.get_fills.return_value = []

    result = reconciler.reconcile(date.today())
    assert result.auto_resolved_count == 1
    assert result.status == "AUTO_RESOLVED"

```

Checklist:

- ☐ Reconciler queries DB correctly
- ☐ Reconciler queries broker API correctly
- ☐ Detects orphaned orders
- ☐ Detects missing fills
- ☐ Categorizes by size (auto vs. manual)
- ☐ Logs results to reconciliation_runs table
- ☐ Sends alerts on manual review items
- ☐ Scheduled to run daily at 4:00 PM ET (after market close)

For DevOps / Infra Engineer (owns Task 1.4 + infrastructure)

Task 1.4: Configuration Management

Create file: config.py

python

```
import os
from dataclasses import dataclass

@dataclass
class TradingConfig:
    # Environment
    ENV: str = os.getenv('ENV', 'dev')

    # Database (required in prod)
    DB_HOST: str = os.getenv('DB_HOST', 'localhost')
    DB_PORT: int = int(os.getenv('DB_PORT', '5432'))
    DB_NAME: str = os.getenv('DB_NAME', 'trading_dev')
    DB_USER: str = os.getenv('DB_USER', 'trading_user')
    DB_PASSWORD: str = os.getenv('DB_PASSWORD') # NO DEFAULT in prod

    # Broker (required)
    BROKER_API_KEY: str = os.getenv('BROKER_API_KEY')
    BROKER_API_SECRET: str = os.getenv('BROKER_API_SECRET')
    BROKER_BASE_URL: str = os.getenv('BROKER_BASE_URL')

    # Risk limits
    MAX_DAILY_LOSS_DOLLARS: float = float(os.getenv('MAX_DAILY_LOSS_DOLLARS', '10000'))
    PER_TRADE_MAX_LOSS_DOLLARS: float = float(os.getenv('PER_TRADE_MAX_LOSS_DOLLARS', '1

    # Strategy allocation
    STRATEGY_ANN_ALLOCATION: float = float(os.getenv('STRATEGY_ANN_ALLOCATION', '0.3'))
    STRATEGY_GRID_ALLOCATION: float = float(os.getenv('STRATEGY_GRID_ALLOCATION', '0.5'))
    BUFFER_ALLOCATION: float = float(os.getenv('BUFFER_ALLOCATION', '0.2'))

    # Logging
    LOG_LEVEL: str = os.getenv('LOG_LEVEL', 'INFO')
    LOG_FILE: str = os.getenv('LOG_FILE', '/var/log/trading/app.log')

    # Async
    ASYNC_WORKERS: int = int(os.getenv('ASYNC_WORKERS', '4'))

    @classmethod
    def load(cls):
        from dotenv import load_dotenv
        if os.path.exists('.env'):
            load_dotenv()

        config = cls()
        config._validate()
        return config
```

```

def _validate(self):
    """Check required fields."""
    if self.ENV == 'prod':
        required = ['BROKER_API_KEY', 'BROKER_API_SECRET', 'DB_PASSWORD']
        for field in required:
            if not getattr(self, field, None):
                raise ValueError(f"Missing required config: {field}")

    # Validate allocations sum to 1.0
    total = (self.STRATEGY_ANN_ALLOCATION +
             self.STRATEGY_GRID_ALLOCATION +
             self.BUFFER_ALLOCATION)
    if abs(total - 1.0) > 0.01:
        raise ValueError(f"Strategy allocations must sum to 1.0, got {total}")

# Load on import
config = TradingConfig.load()

# Usage in other modules:
# from config import config
# db = connect(config.DB_HOST, config.DB_PORT, config.DB_NAME)

```

Create file:

bash

```
# Environment
ENV=dev

# Database
DB_HOST=localhost
DB_PORT=5432
DB_NAME=trading_dev
DB_USER=trading_user
DB_PASSWORD=change_me_in_prod

# Broker
BROKER_API_KEY=your_api_key_here
BROKER_API_SECRET=your_api_secret_here
BROKER_BASE_URL=https://api.broker.example.com

# Risk
MAX_DAILY_LOSS_DOLLARS=10000
PER_TRADE_MAX_LOSS_DOLLARS=1000

# Strategy
STRATEGY_ANN_ALLOCATION=0.30
STRATEGY_GRID_ALLOCATION=0.50
BUFFER_ALLOCATION=0.20

# Logging
LOG_LEVEL=INFO
LOG_FILE=/var/log/trading/app.log

# Async
ASYNC_WORKERS=4
```

Deployment:

```
bash

# Create .env in container at runtime
docker run -e ENV=prod -e BROKER_API_KEY=$BROKER_KEY ... trading-system
```

Checklist:

- ☐ All hardcoded values moved to config.py
- ☐ .env.example committed to repo
- ☐ .env added to .gitignore
- ☐ Config validates on load
- ☐ Can be deployed to dev/test/prod with different .env
- ☐ Secrets never logged (use logging.handlers.filter)

- ☐ CI/CD passes .env to containers safely (not in Dockerfile, use secrets)
-

Minimum Viable Completion Criteria

By Friday end-of-day Week 1, you need:

- State Machine Document** (`execution_state_machine.md`)
 - ☐ At least 5 states defined (CREATE, QUEUED, SENT, ACK, FILLED, FAILED, DEAD_LETTER)
 - ☐ Every state has transition rules
 - ☐ Current implementation status for each state (MISSING/PARTIAL/COMPLETE)
- Current vs Target Architecture** (`architecture_current_vs_target.md`)
 - ☐ Current state: describe actual code structure (monolithic or modular?)
 - ☐ Target state: draw ideal microservices layout
 - ☐ Gap analysis table with effort estimates
- Capital Safety Baseline** (`capital_safety_requirements.md`)
 - ☐ Kill switch spec (hard stop, idempotent, logged)
 - ☐ Broker reconciliation spec (daily, detect orphaned orders)
 - ☐ DB audit trail spec (immutable event log)
 - ☐ Capital allocation spec (per-strategy limits)

All three documents should be:

- ☐ Clear and unambiguous
 - ☐ Signed off by CTO + risk lead
 - ☐ Ready to hand to developers on Monday Week 2
-

Red Flags (Escalate Immediately)

If you encounter any of these, stop and ask for help:

- Circular Dependencies:** Module A imports B, B imports A → refactor before proceeding
- Broker API Changes:** Planned changes to broker API → coordinate timeline with their team
- Missing Historical Data:** Can't backtest because no price history → find data source now
- Database Performance:** Kill switch + DB queries taking >100ms → optimize or scale DB
- Risk Disagreement:** Risk team says kill switch isn't sufficient → escalate to CRO before proceeding

6. **External Dependency:** Waiting for broker to add API endpoint → add buffer time to schedule
-

Questions to Ask Stakeholders

Before starting Phase 1 (Week 2), make sure you can answer:

1. **Business** (CFO / Risk Officer)
 - What's the minimum daily loss that triggers automatic stop? (\$10k? \$50k?)
 - How quickly can we recover from a kill switch event? (24h? 1 hour?)
 - What's acceptable latency between our system and broker? (10ms? 100ms?)
 2. **Technical** (CTO)
 - Is there existing monitoring/alerting infrastructure? (DataDog? CloudWatch?)
 - Are there security requirements? (PII in logs? IP whitelisting?)
 - Should we use existing message queue (Kafka?) or build custom async?
 3. **Operations** (Ops Lead)
 - Who's on-call during market hours?
 - What's the runbook if reconciliation fails?
 - How do we deploy zero-downtime updates?
-

Resources

- **This Repo:** Clone/fork it to your GitHub
 - **Tools:**
 - Docker (for environment consistency)
 - PostgreSQL 14+ (DB)
 - Python 3.10+ (code)
 - asyncio (async queue)
 - FastAPI (WebSocket dashboard)
 - **Testing:** pytest, pytest-asyncio, testcontainers (PostgreSQL in Docker)
 - **Documentation:** Markdown files in this repo
 - **Monitoring:** [Choose 1] DataDog, Prometheus+Grafana, CloudWatch, New Relic
-

Good luck! You've got this. 🚀